

PROCESO DE DESARROLLO DE SOFTWARE · UADE 2026

Primero el negocio. Después, la arquitectura.

LogiSmart es una plataforma SaaS de logística para PyMEs. La recorreremos capa por capa: cada decisión de diseño existe para sostener una necesidad concreta del negocio.

Rosario Presedo
LU 1146894

Simón Ottati
LU 1155931

Comisión Belgrano
UADE · 2026

5
capas

5
atributos

36
patrones

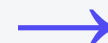
190
tests ✓

Una PyME que hoy gestiona su logística a mano

HOY · FRAGMENTADO

Planillas, WhatsApp y memoria

- Pedidos del e-commerce cargados a mano, uno por uno.
- Rutas armadas «a ojo», sin tráfico ni ventanas horarias.
- El cliente final llama para preguntar «¿dónde está mi paquete?».



LOGISMART

CON LOGISMART · CENTRALIZADO

Una sola plataforma SaaS multitenant

- Importa pedidos del e-commerce automáticamente.
- Arma rutas óptimas según tráfico real y capacidad de flota.
- El cliente final sigue su envío en un mapa, con ETA real.

▶ **Misión:** convertir la logística de un centro de costos en una ventaja competitiva. **Visión:** una solución todo-en-uno, económica y fácil de implementar, que una sola plataforma sirva a muchas PyMEs —cada una su propio «tenant».

Tres casos de uso son el corazón del MVP

CU-01

Importar pedidos

Sincroniza con TiendaNube / MercadoShops y trae las órdenes pendientes — adiós a la carga manual.

CU-03

Planificar rutas óptimas

Analiza pedidos, vehículos y tráfico real, y devuelve la secuencia de paradas más eficiente respetando ventanas horarias.

CU-07

Seguir el envío en vivo

El cliente final entra por un enlace, ve al chofer en un mapa y el ETA calculado según el avance real de la ruta.

ACTORES INTERNOS

Operador Logístico
crea envíos · asigna rutas · gestiona flota · reportes

Admin. PyME / Empresa
crea envíos · consulta reportes

Admin. de Plataforma (LogiSmart)
administra tenants · todos los permisos

Transportista
ejecuta rutas · actualiza posición GPS

Cliente Final
sigue su envío en tiempo real · CU-07



LogiSmart

SAAS
MULTITENANT



SISTEMAS EXTERNOS

CARRIERS

DHL

FedEx

UPS

PAGOS

PayPal

Stripe

PROPUESTOS · NO IMPLEMENTADOS

Notificaciones

Mapas

Geolocalización

Cinco atributos de calidad: la brújula del diseño

• CRÍTICA Alta disponibilidad Si LogiSmart se cae, se frena la operación de la PyME. Objetivo 99,9% de uptime. → Aplicación	• ALTA Aislamiento de datos La info de cada PyME debe ser inaccesible para otros abonados del SaaS. → Presentación	• ALTA Interoperabilidad Integración fluida con e-commerce, mapas y pagos heterogéneos, sin contaminar el dominio. → Infraestructura	• MEDIA - ALTA Elasticidad Adaptar recursos según demanda y región (AR / BR) para optimizar costos. → Infra + Dominio	• MEDIA Eficiencia de respuesta Cálculo de rutas y consultas de tracking que respondan rápido. → Persistencia
--	---	---	--	--

▶ Cada patrón sostiene uno de estos atributos — **no «porque sí»**. Y los atributos **compiten**: diseñar es decidir estas tres tensiones.

Seguridad ⇄ Performance El Controller valida permisos en cada operación (paga latencia); Proxy + caché + Lazy Load la recuperan en Persistencia.	Flexibilidad ⇄ Simplicidad Asumimos más clases que un condicional — solo donde hay un punto de variación real (tarifas, estados, proveedores).	Anticipar ⇄ Sobre-ingeniería Protegemos los puntos de variación real; cada abstracción nace de un problema concreto, no de anticipar el futuro.
--	--	---

Cada atributo de calidad vive en una capa

FLUJO DE UNA PETICIÓN

Presentación

01 Recibe la petición, autoriza y delega. Sin lógica de negocio.

→ Aislamiento de datos

Aplicación

02 Orquesta los casos de uso entre subsistemas, sin que nada quede a medias.

→ Alta disponibilidad

Dominio • el corazón

03 Las entidades y sus invariantes. Todo gira en torno a **Envío** y su ciclo de vida.

→ Trazabilidad en tiempo real

Persistencia

04 Aísla el almacenamiento del dominio. El dominio nunca toca SQL.

→ Eficiencia de respuesta

05

Infraestructura

Capa transversal: aloja patrones de comportamiento, estructurales y de persistencia, además de la integración con sistemas externos. La usan todas las demás.

→ Interoperabilidad

→ Elasticidad

Un SaaS multi-tenant para PyMEs logísticas

ACTORES · 5 ROLES

Admin. Plataforma

LogiSmart · acceso total

Operador Logístico

PyME · usuario principal

Admin. Empresa

PyME · dueño / gerente

Transportista

PyME · chofer

Cliente Final

Externo · CU-07

▶ Cada PyME es un **tenant** con su **Suscripción**. Los permisos salen de la matriz 5×5 del enum `Ro1`.

ENVÍO · LA ENTIDAD CENTRAL

Ciclo de vida (State)
CONFIRMADO → EN_TRANSITO
→ EN_REPARTO → ENTREGADO
↳ CANCELADO · RETENIDO desde intermedios

Tipos de proceso (Template Method)

- Nacional** · peso × \$100
- Internacional** · peso × \$250 + \$1.000
- Urgente** · peso × \$180 + \$500

Estrategias de costo (Strategy · 6)
Peso · Distancia · Urgencia · Volumen · Fijo · Híbrida

Servicios adicionales (Decorator)
GPS tracking +\$50 · Seguro +15 %

FLOTA · 3 VEHÍCULOS · 2 REGIONES

Moto
100 kg
\$0.90/km

Auto
500 kg
\$1.00/km

Camión
5.000 kg
\$1.80/km

AR Argentina

- Default:** Auto
- IVA** 21 %
- Google Maps
- $(\$100 + \text{peso} \times \$15) \times 1.21$

BR Brasil

- Default:** Moto
- ICMS** 12 %
- HERE Maps
- $(\$80 + \text{peso} \times \$12) \times 1.12$

▶ `crearFactory(region)` — el único punto del sistema con fábricas concretas. El resto del código solo ve `LogiSmartFactory`.

Que ninguna PyME vea los datos de otra

ATRIBUTO Aislamiento de datos · seguridad (alta)

- PERMISOS ANTES QUE NADA

>

Controller

Una sola puerta de entrada que valida quién pide qué, y delega.

- PERMISOS SIN IF FRÁGILES

>

Rol · Polymorphism

El acceso de cada rol resuelto con polimorfismo, no con condicionales.

Multitenant por diseño

El control de acceso vive arriba para que el **dominio quede testeable en aislamiento** — y para que los datos de un tenant nunca crucen hacia otro.

← expandí una tarjeta para ver el fragmento en código

25 booleanos dispersos, un enum centralizado

25
booleanos
en 5 clases

→

1
enum · fuente
de verdad única

▶ Un permiso nuevo: antes, **5 archivos** a editar. Ahora, **una columna más en el enum**. El cambio se realizó con la batería de tests en verde — sin miedo a romper nada.

Antes — duplicado en 5 clases

```
// AdminEmpresa.java
private boolean puedeCrearEnvio    = true;
private boolean puedeAsignarRuta   = false;
private boolean puedeVerReportes   = true;
// ídem en Transportista, ClienteFinal...
// → 5 clases × 5 permisos = 25 booleans
```

Después — GRASP Expert centralizado

```
// Rol.java — única fuente de verdad
//           crear asig rep  flota admin
OPERADOR    (true,  true,  true,  true,  false),
ADMIN_EMPRESA (true,  false, true,  false, false),
ADMIN_PLATAFORMA (true, true,  true,  true,  true ),
CLIENTE     (false, false, false, false, false),
TRANSPORTISTA (false, false, false, false, false);
```


Orquestar el caso de uso sin que nada quede a medias

ATRIBUTO Alta disponibilidad · confiabilidad (crítica)

- UNA LLAMADA PARA UNA OPERACIÓN COMPLEJA

>

Facade

Una interfaz simple sobre cinco subsistemas que se coordinan en orden.

- FALLAR BARATO Y TEMPRANO

>

Chain of Responsibility

Validación en cadena que corta en el primer rechazo.

Orquesta, no decide

Es la capa que más se apoya en GRASP: **bajo acoplamiento y alta cohesión**. La lógica real siempre baja al dominio.

+ En esta capa también: **Command** · **Mediator** · **Template Method**

Un patrón, una deuda técnica, una corrección

ANTES · DEUDA

```
// Firma frágil: cada campo nuevo
// obliga a modificar TODOS los nodos
boolean validar(Envio envio, Cobro cobro) {
    ...
}
```

Con el quinto validador (Hito 10) la firma creció. Agregar un campo cascadea a toda la cadena — la deuda acumula intereses.

DESPUÉS · LIQUIDADA

```
// ContextoValidacion = Value Object
// Los nodos no cambian cuando el
// contexto crece
boolean validar(ContextoValidacion ctx) {
    ctx.getEnvio(); ctx.getCobro();
}
```

`ContextoValidacion` encapsula el contexto completo. Los validadores son **Protected Variations**: inmunes a cómo crezca el contexto mañana.

▶ Agregar el quinto validador fue una clase nueva. Liquidar la deuda fue un refactor acotado a un solo punto. **Open/Closed en acción** — la cadena creció sin tocar los nodos existentes.

Envío, el corazón del negocio



Seleccioná un patrón para ver su rol en [Envío](#) — o seguí al slide siguiente para ver State, Observer y Memento en detalle.

Que el cliente final siga su envío en tiempo real

PROMESA ESTRELLA Trazabilidad + robustez del envío

- TRANSICIONES VÁLIDAS, SIEMPRE >

State

El envío evoluciona solo, siguiendo su máquina de estados.

- AVISAR A TODOS AL INSTANTE >

Observer

Al cambiar de estado, el envío notifica a sus suscriptores.

- DESHACER Y AUDITAR >

Memento

Una foto inmutable del envío para restaurar o comparar.

+ El dominio también modela:

Composite

Strategy

Visitor

Decorator

Interpreter

Respuestas rápidas sin que el dominio toque SQL

ATRIBUTO Eficiencia de respuesta + consistencia

- EL DOMINIO PIDE OBJETOS, NO FILAS >

Repository + Data Mapper

Acceso uniforme que esconde si el dato viene de memoria o de SQL.

- NO PEGARLE 10 VECES A LA BASE >

Proxy + Lazy Load

Cachea consultas y difiere la carga hasta que se necesita.

- TODO O NADA >

Unit of Work

Agrupar los cambios y los confirma en una transacción atómica.

► **Decisión clave:** reutilizar el dominio existente — el mapper traduce solo el subconjunto persistente de cada entidad.

Una decisión de H7 que no sabía lo que hacía por H13

— HITO 7 — BUILDER PARA EL DOMINIO

El Builder se diseñó para construir `Envio` con 12 atributos de forma legible. La motivación era el dominio — no la persistencia.

```
// FabricaDeEnvios.java — H7
Envio envio = new Envio.EnvioBuilder("e-001")
    .origen("BUE").destino("COR")
    .peso(12.5).tipo("EXPRESS")
    .build();
```

— HITO 13 — EL MISMO BUILDER, PARA PERSISTENCIA

El Data Mapper necesitaba reconstruir `Envio` desde la base de datos. El Builder ya existía. **No hubo refactoring.**

```
// EnvioMapperSQL.java — H13
Envio envio = new Envio.EnvioBuilder(row.id)
    .origen(row.origen).destino(row.destino)
    .peso(row.peso).tipo(row.tipo)
    .build(); // ← mismo Builder, 6 hitos después
```

▶ Lo sembrado en el Hito 7 se cosechó en el Hito 13. Así funciona el enfoque iterativo: las decisiones de diseño tempranas pagan dividendos iteraciones después — si están bien pensadas.

Hablar con el mundo externo y operar en dos países

ATRIBUTOS Interoperabilidad + Elasticidad regional

- INTEGRAR SIN CONTAMINAR EL DOMINIO



Adapter

Traduce APIs externas incompatibles a un contrato propio.

- COHERENCIA POR REGIÓN (AR Y BR)



Abstract Factory

Familias de objetos compatibles, nunca piezas mezcladas.

- 3 TIPOS × 4 FORMATOS → 7 CLASES, NO 12



Bridge

Separa qué se reporta del cómo se formatea.

- 10.000 ENVÍOS, NO 30.000 OBJETOS



Flyweight

Comparte ubicaciones inmutables que se repiten muchísimo.

+ También transversal: **Factory Method** · **Singleton**

Dos países, dos reglas de negocio — nunca mezcladas

AR FactoryArgentina

- **Vehículo:** Auto
- **Calculador:** IVA 21% sobre base
- **Mapas:** Google Maps Argentina

```
new LogiSmartFactoryArgentina()
    .crearVehiculo()           // → Auto
    .crearCalculador()         // → IVA 21%
    .crearProveedorMapas()     // → GoogleMaps AR
```

BR FactoryBrasil

- **Vehículo:** Moto
- **Calculador:** ICMS 12% sobre base
- **Mapas:** HERE Maps Brasil

```
new LogiSmartFactoryBrasil()
    .crearVehiculo()           // → Moto
    .crearCalculador()         // → ICMS 12%
    .crearProveedorMapas()     // → HERE Maps BR
```

! **Sin el patrón:** un auto argentino con cálculo brasileño (ICMS 12%) produce costos incorrectos silenciosamente. No hay error en compilación — solo un número mal.

▶ La Factory garantiza que los objetos de una región son **siempre coherentes entre sí**. Nunca un Auto argentino con HereMaps brasileño.

Composite facilita agregar nodos · Visitor facilita agregar operaciones

Composite — la estructura

```
CentroRegional           // Composite
├─ SucursalEntrega       // Leaf
└─ SucursalEntrega       // Leaf

aceptar(visitor) ← punto de unión
```

- ✓ Nuevo nodo → 1 clase nueva, árbol listo
- ✓ Cliente trata árbol y hoja igual (*transparencia*)
- ✗ Nueva operación → habría que tocar la interfaz

Visitor — las operaciones

```
VisitorCalculoOcupacion
VisitorCostoOperativo
VisitorBusquedaCriticos
  ↓ double dispatch
visitar(CentroRegional centro)
```

- ✓ Nueva operación → 1 Visitor nuevo, nodos intactos
- ✓ 3 operaciones sumadas sin modificar el árbol
- ✗ Nuevo tipo de nodo → actualizar todos los Visitors

▶ La estructura del árbol de centros es **estable** (regional + puntos de entrega). Las *operaciones* crecieron hito a hito. Por eso Visitor fue la decisión correcta: el árbol absorbió tres nuevas operaciones sin tocar un solo nodo existente.

Una petición que cruza las cinco capas

1

Presentación

`Controller` recibe ·
`Rol.puedeCrear()`
autoriza antes de delegar
— sin lógica de negocio

`Controller`

`Info` `Expert`

2

Aplicación

`Facade` orquesta los
subsistemas · `Chain`
valida en orden de costo,
fail-fast

`Facade`

`Chain`

3

Infraestructura

`Factory` crea el
`Envio` regional ·
`Adapter` traduce al
protocolo del carrier

`Abstract`
`Factory`

`Adapter`

4

Dominio · el corazón

`Envio.cambiarEstado()`
transiciona con `State` · el
cambio dispara `Observer`
automáticamente

`State`

`Observer`

5

Persistencia

`Proxy` cachea las
lecturas frecuentes ·
`Repository` guarda
objetos sin que el
dominio toque SQL

`Proxy`

`Repository`

Planificar la ruta óptima: dos Strategy encadenadas

1

Controller

Recibe el pedido de planificación y delega sin elegir el algoritmo — la firma `planificarRuta()` nunca cambia

GRASP Controller

2

AsignadorDeVehiculos

Strategy 1 — elige el vehículo óptimo según el criterio configurado

Strategy

PorCapacidad

PorDisponibilidad

3

CalculadorDeRuta

Strategy 2 — elige la ruta óptima entre candidatas con criterio intercambiable

Strategy

MasBarata

MasCercana

MenosCongestionada

4

Ruta

Information Expert — conoce su propio recorrido y calcula **distancia** y **costo total**

Info Expert



Cambiar el criterio de ruta = enchufar otra `Strategy` en el paso 3, **sin tocar el Controller ni la firma del método.**

asignarRuta : seis patrones en un solo método

Todos ocurren dentro de `asignarRuta()` en el Controller — no están dispersos por el sistema

1 Verificar permiso

`Rol.puedeAsignar()` — el dominio nunca ve la lógica de seguridad

Information Expert

2 Snapshot de seguridad

`Memento.snapshot()` guarda el estado del envío antes de cualquier cambio

Memento

3 Elegir vehículo

`AsignadorDeVehiculos` selecciona por capacidad o disponibilidad

Strategy

4 Calcular ruta

`CalculadorDeRuta` elige la ruta óptima entre candidatas

Strategy

5 Transición de estado

`envio.setEstado(EN_TRÁNSITO)` — la transición es validada por el estado actual

State

6 Notificar + persistir

`Observer` propaga el ETA · `Repository` + `UnitOfWork` confirman juntos

Observer

Repository

Unit of Work

Seguimiento en vivo: State dispara Observer

1

Transportista reporta

Informa su posición GPS. El servicio de tracking se **apila sobre** `Envio` con Decorator — sin modificar la clase base

Decorator

2

`Envio.cambiarEstado()`

State valida que la transición sea legal · luego llama a `notificarObservadores()` — el envío no sabe quién escucha

State

EN_TRÁNSITO

EN_REPARTO

ENTREGADO

3

Observer · tres suscriptores

`notificarObservadores()` propaga el evento a todos:

- **Dashboard** — refresca el mapa y el ETA
- **Cliente final** — email o SMS con ETA actualizado
- **Centro** — actualiza su ocupación y capacidad

Observer

PROMETIMOS

- CU-01 Importar pedidos del e-commerce
- CU-03 Planificar rutas óptimas
- CU-07 Seguir el envío en vivo

CUMPLIMOS

13	5	36	229	190 ✓
HITOS	CAPAS	PATRONES	CLASES	TESTS

Primero el negocio.
Después, la arquitectura.

¿Preguntas?